

## **Abstract**

The inverted pendulum system was demonstrated for this project, which represents a fundamental problem in control systems. The system set up consisted of a cart, where a pendulum was mounted on top with the objective being to keep the pendulum stable while being exposed to environmental factors and forces. The project employs Model Based Systems Engineering (MBSE)- done through the Papyrus software using SysML to model the structural and behavioral components of the system. Alongside with mathematical modeling of the Lagrangian method done through Python to investigate the system's behavior. The simulation captured real-world conditions by incorporating noise, friction, air damping into the environmental factors into the modeling conditions. The project focused on comparing two types of controllers: Proportional- Integral- Derivative (PID) and full state feedback/pole placement controller. Performance metrics such as settling time, overshoot, and stability under noise and disturbances are used to benchmark the controllers. It was found that the PID controller had better performance considering the settling time and overshoot compared to the pole placement controller. However, the pole placement controller worked better under the noise.

# **TABLE OF CONTENTS**

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>Abstract</b>                                      | <b>2</b>  |
| <b>TABLE OF CONTENTS</b>                             | <b>3</b>  |
| <b>1.0 INTRODUCTION</b>                              | <b>4</b>  |
| <b>2.0 SIMPLIFIED DESIGN PROCESS</b>                 | <b>4</b>  |
| 2.1 Problem Definition                               | 4         |
| 2.2 Solution Formulation                             | 5         |
| 2.3 Modeling and Prototyping                         | 6         |
| 2.3.1 MBSE                                           | 6         |
| 2.3.2 Mathematical Modeling                          | 9         |
| 2.3.3 Simulated Modeling with Controller             | 12        |
| 2.3.4 Benchmarking and Performance Metrics           | 16        |
| <b>3.0 CONCLUSION &amp; RECOMMENDATIONS</b>          | <b>18</b> |
| <b>4.0 REFERENCES</b>                                | <b>18</b> |
| <b>5.0 APPENDIX</b>                                  | <b>19</b> |
| 5.1 Task 2- System Exposure to Environmental Factors | 19        |
| 5.2 Task 3- PID Controller                           | 22        |
| 5.3 Task 3- Pole Placement Controller                | 26        |

## 1.0 INTRODUCTION

The project demonstrated the modeling and simulations of a pendulum mounted on a moving cart exposed to an impulse force. Due to being unstable as it is an inverted pendulum, the system required a continuous application of control and feedback to remain in an upright position. As the system was already difficult to balance, an additional challenge was imposed by the added disturbances in the system (impulse force, noise, friction, air resistance) causing extreme sensitivity in the system. Therefore, the controller has to be designed to react quickly to changes and appropriately apply the feedback received from the sensors.

The problem was first modeled using MBSE to understand the nature of the problem and the components of it. SysML was used to model the structure and interactions of the system. Following Python scripts which were used for simulation and design of the two (2) controllers- PID and pole placement.

This report provided the steps taken to design and simulate the inverted pendulum system through the simplified design process, which takes into account the problem definition, formulation of a solution, mathematical modeling and prototyping, and performance benchmarking.

## 2.0 SIMPLIFIED DESIGN PROCESS

### 2.1 Problem Definition

The issue with the inverted pendulum system was that due to the natural effects of gravity and environmental factors such as air drag, friction- the system finds difficulty in maintaining the pendulum in an upright position. As a result, continuous feedback was required on the system to keep stabilization conditions. The project looked into applying a force on the pendulum and based on controller information the system was to be developed to stabilize itself.

The main challenges of this problem included:

**Nonlinearity:** The nature of the system is nonlinear, therefore, to formulate solutions either the system had to be transformed to linearization or the controller had to be adjusted to provide nonlinear control.

**Unstability:** Without an active control system, the pendulum will fall, making it a difficult system to stabilize.

Sensor Noise: Sensor readings are subject to noise depending on resolution and sensitivity of the instrument, which can affect the controller's performance. Therefore, filtering techniques need to be implemented to handle noisy measurements.

The system must meet the following performance criteria:

Minimize settling time: The system should stabilize the pendulum as quickly as possible.

Minimize overshoot: The angle of the pendulum should sway within a certain range and slowly minimize this range.

Fast reaction to disturbance: The controller should handle external disturbances and measurement noise quickly to reduce system sensitivity of disturbance.

Table 1: Problem Definition Chart

| Product Characteristics           | Functional Requirements                                | Constraints                  | Performance Metric       |
|-----------------------------------|--------------------------------------------------------|------------------------------|--------------------------|
| System must maintain stability    |                                                        | Mass of pendulum is 50 grams | Settling time of system  |
| System must be quick              | Respond to external disturbances and quickly stabilize | Length of pendulum is 40 cm  | Overshoot of system      |
| System must be resistant to noise | System must stay upright despite noise                 |                              | Oscillation of system    |
| System must be lightweight        |                                                        |                              | Oscillation due to noise |

## 2.2 Solution Formulation

To address the problem, the inverted pendulum system was modeled using real world variables, such as noise, friction, and air resistance.

Modeling the system dynamics: To model the system, Lagrangian method was used to obtain nonlinear equations of motion. As shown below, the equations were linearized using small angle approximation. However, to run the Python script the equations were solved using the *scipy* library to then simulate the system's behavior.

Simulating sensor noise: Gaussian noise was added to the readings of the pendulum position and angle as expected from sensors. A low pass filter was then added to remove the noise from the readings and provide a reduced effect of noise on the system.

**PID Controller:** A controller that adjusts the force applied to the cart based on the error between the desired and actual pendulum angle.

**Pole Placement Controller:** A state-space controller that places the poles of the in desired locations to achieve stability.

**Benchmarking and performance evaluation:** The performance of the controllers is evaluated based on key metrics such as settling time, overshoot, and robustness to noise.



Figure 1.0: Mind Mapping for Formulating Solution

## 2.3 Modeling and Prototyping

### *2.3.1 MBSE*

Model Based Systems Engineering (MBSE) was done using SysML on the Papyrus software. There were 6 diagrams created to capture the system architecture, components and interactions.

- **Block Definition Diagram (BDD):** This diagram was used to define the four (4) components of the system: cart, pendulum, sensor, and controller. Along with the defining properties of each component, such as mass, velocity, force, angle. The diagram provided a holistic view of which signals interacted/were a part of each component.

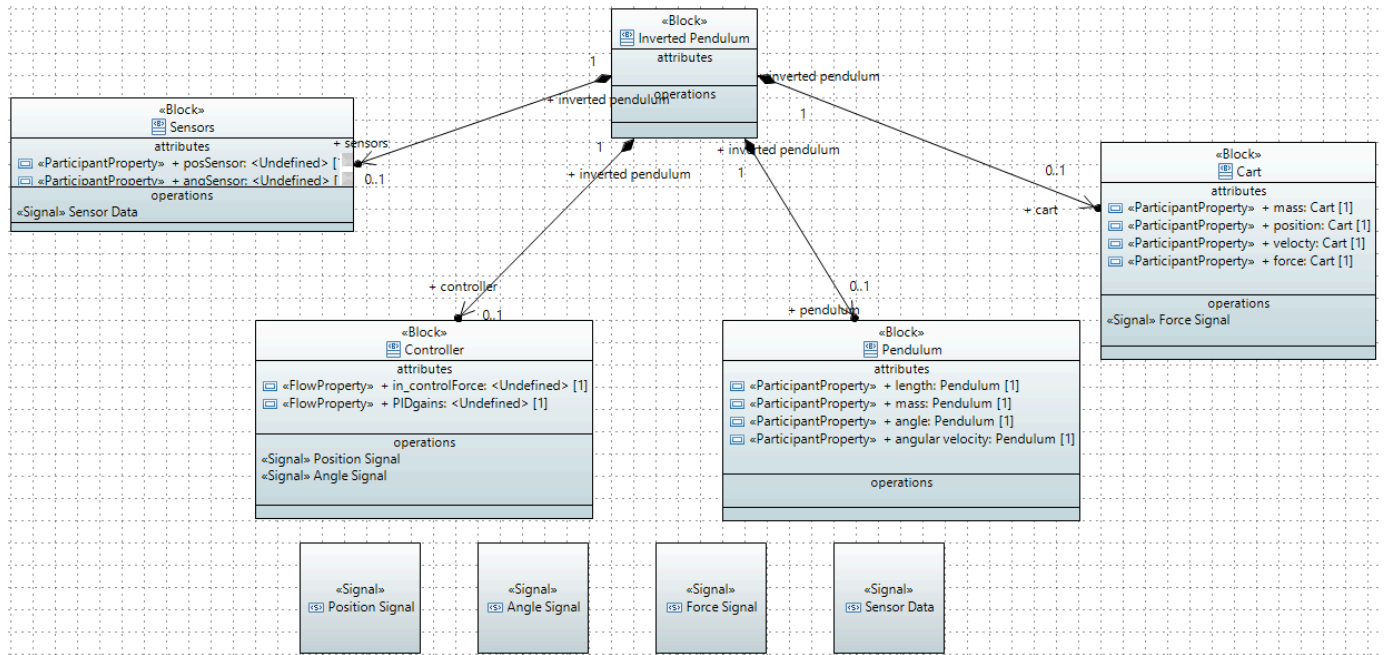


Figure 2.0: Block Diagram of System

- Internal Block Diagram (IBD):** This diagram was used to show how the components above interact within the system including all signal commands. The core component of the system relies on signals received and transmitted by the sensor. As the sensor receives the signals from cart and pendulum and provides it back to the controller. The controller then provides updated angle and position data for it to stabilize.

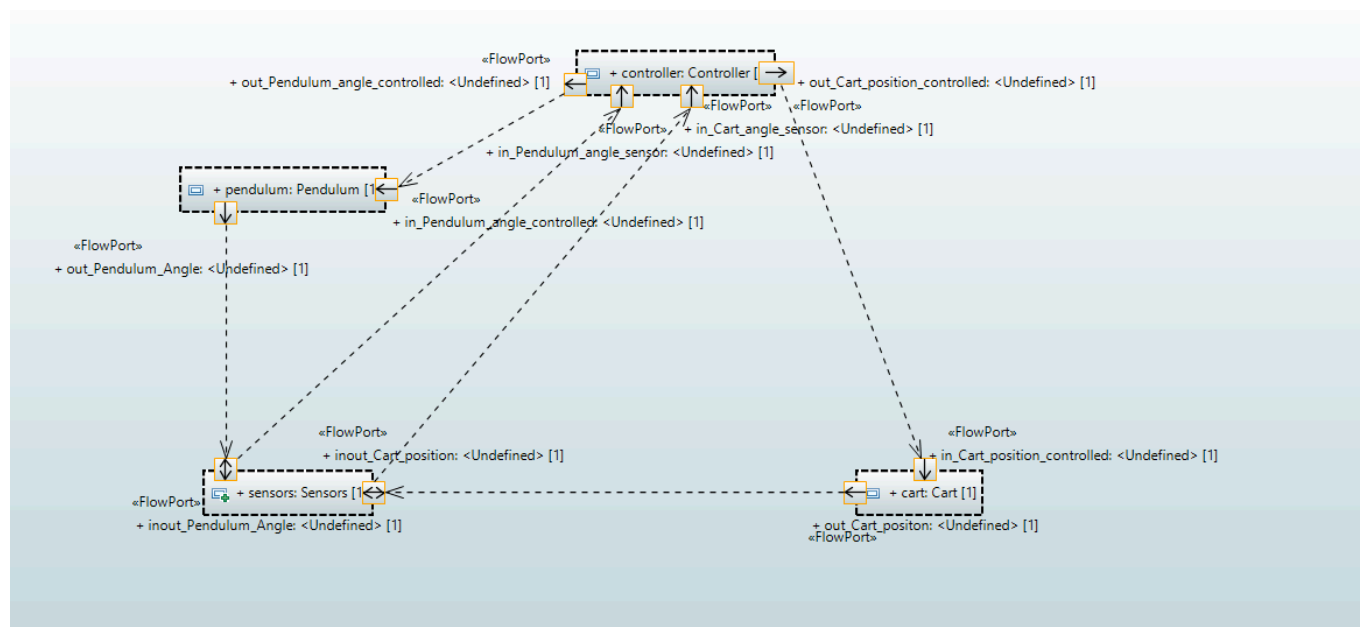


Figure 2.1: Internal Block Diagram of System

- Activity Diagram: This diagram showed a sequence of activities that was done by the system in order for it to complete its controlling task.

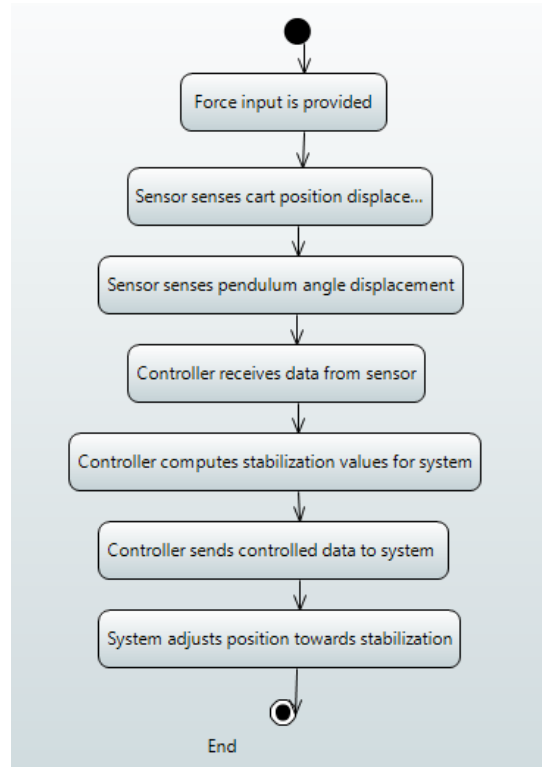


Figure 2.2: Activity Diagram of System

- Use Case Diagram: This diagram provided a high-level view of the system's functionalities from the perspective of users or other systems. It highlighted the interactions between the users and the system, specifying key use cases such as "Monitor Pendulum Position," "Adjust Cart Position," and "Receive Sensor Feedback."

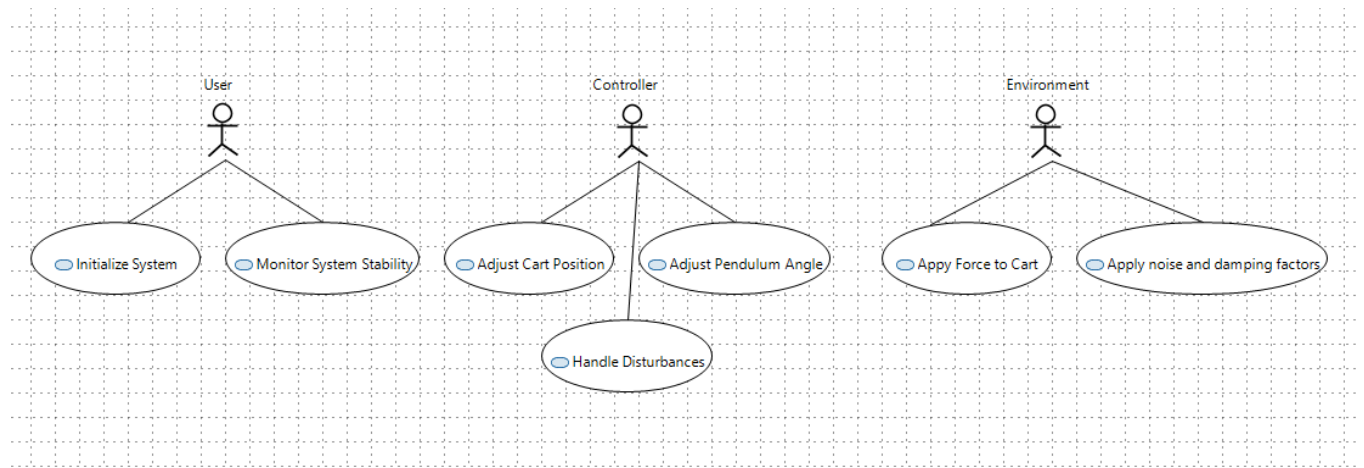


Figure 2.3: Use Case Diagram of System

- **Sequence Diagram:** The sequence diagram describes how the system components communicate over time, particularly focusing on the feedback loop between the sensors and the controller.

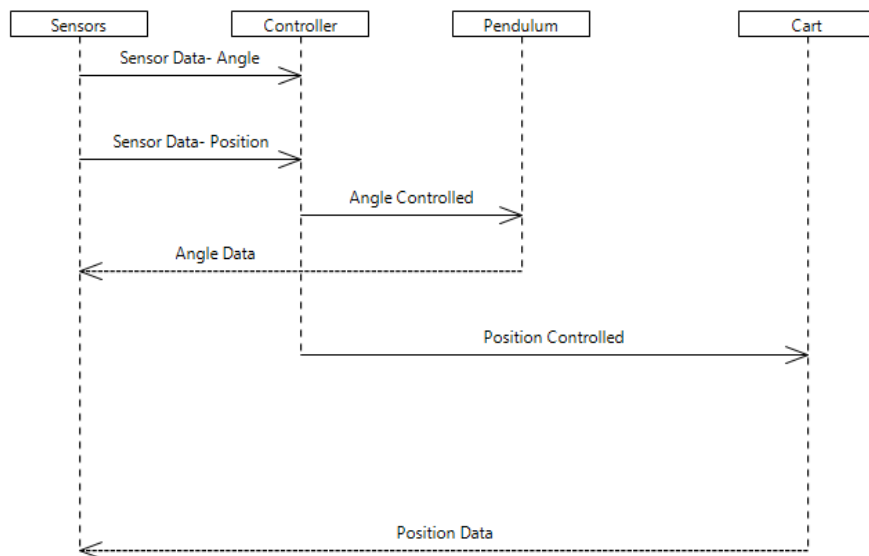


Figure 2.3: Sequence Diagram of System

### 2.3.2 Mathematical Modeling

To mathematically model the inverted pendulum system Lagrangian equations were used, where it is represented overall as  $L = KE - PE$ . The equation takes into account the potential and kinetic energy of the system. For this system the equation was represented as follows:



$$L = \frac{1}{2}m_1 v_1^2 + \frac{1}{2}m_2 v_2^2 - mgl\cos\theta$$

Where,

$m_1$  is the mass of the cart

$m_2$  is the mass of the pendulum

$v_1$  is the velocity of the cart

$v_2$  is the velocity of the pendulum

Differentiating with respect to time for the velocity of the cart and pendulum:

$$v_1^2 = \dot{x}^2$$

$$v_2^2 = \left(\frac{d}{dt}(l\cos\theta)\right)^2 + \left(\frac{d}{dt}(x - l\sin\theta)\right)^2$$

The kinetic energy can be expressed as:

$$KE = \frac{1}{2}(m_1 + m_2)\dot{x}_1^2 - m_2 l \dot{x}_1 \dot{\theta} \cos(\theta) + \frac{1}{2}m_2 l^2 \dot{\theta}^2$$

The final expression for the Langrangian expression is as follows:

$$L = \frac{1}{2}(m_1 + m_2)\dot{x}_1^2 - m_2 l \dot{x}_1 \dot{\theta} \cos(\theta) + \frac{1}{2}m_2 l^2 \dot{\theta}^2 - m_2 gl\cos(\theta)$$

To find the equations of motion, the Lagrange equations are defined as:

$$\frac{d}{dt}\left(\frac{\delta L}{\delta \dot{x}_1}\right) - \frac{\delta L}{\delta x_1} = F$$

$$\frac{d}{dt}\left(\frac{\delta L}{\delta \dot{\theta}}\right) - \frac{\delta L}{\delta \theta} = 0$$

Where,

$$\frac{\delta L}{\delta \dot{x}_1} = (m_1 + m_2)\dot{x}_1 - m_2 l \dot{\theta} \cos(\theta)$$

$$\frac{d}{dt}\left(\frac{\delta L}{\delta \dot{x}_1}\right) = (m_1 + m_2)\ddot{x}_1 - m_2 l \ddot{\theta} \cos(\theta) + m_2 l \dot{\theta} \sin(\theta)$$

$$\frac{\delta L}{\delta x_1} = 0$$

$$\frac{\delta L}{\delta \theta} = -m_2 l \dot{x}_1 \sin(\theta) + m_2 l^2 \ddot{\theta}$$

$$\frac{d}{dt}\left(\frac{\delta L}{\delta \dot{\theta}}\right) = -m_2 l \ddot{x}_1 \sin(\theta) + m_2 l \dot{x}_1 \cos(\theta) + m_2 l^2 \ddot{\theta}$$

$$\frac{\delta L}{\delta \theta} = m_2 l \dot{x}_1 \sin(\theta) + m_2 gl\sin(\theta)$$

Simplifying the equations provides the following nonlinear equations:

$$(m_1 + m_2)\ddot{x} + B\dot{x} - m_2 l \ddot{\theta} \cos(\theta) + m_2 l \dot{\theta}^2 \sin(\theta) = F$$

$$m_2 l (-g \sin(\theta) - \ddot{x} \cos(\theta) + \dot{x} \dot{\theta}) = 0$$

Linearization of the equation can be done using small angle approximation, where,  $\theta \approx 0$

The final equations are then:

$$m_1 \ddot{x}_1 + B\dot{x} - mg\theta = F$$

$$m_2 l \ddot{\theta} - (m_1 + m_2)g \theta + B\dot{x} = 0$$

In the modeling and simulation B was considered to be the combination of the damping caused by friction and air resistance. While simulating the damping coefficient was chosen to be 0.5.

The code of the system is explained below:

#### 1) Set Up of System

- The code started with importing the relevant libraries, numpy for computations, matplotlib for graphing visualizations, scipy to solve the nonlinear equations, place\_poles to compute the state space control design, and FuncAnimation to run the animation of the system.
- Parameters were defined for the mass of cart and pendulum, length of pendulum, gravity and damping (which considered both friction and air resistance).
- The nonlinear equations of motion were defined as shown above and calculated within a function named equations
- The simulation was set to run for 60 seconds with initial conditions of no displacement for cart position, no velocity for cart, 180 degrees for pendulum angle, no angular velocity for pendulum.
- An external force of 5N was defined for the system

#### 2) Set Up of Noise within System

- A function was created which created noise within the system at 0.02
- The noise was added to position and angle readings

#### 3) Set Up of Low Pass filter Band

- A function containing an array was made and the position and angle values were filtered for the noise that was initially added.

#### 4) Animation and Graphing

- The simulation parameters were restricted within the displacement of -1.0 and 1.0 to prevent the cart from going off the screen

- The graphing parameters were set by defining the axis labels, line colors, and legend.
- A function was created to obtain the pendulum position in x,y coordinates
- The cart and pendulum were created by creating objects which defined the characteristics of the system.
- The animation updates were done through creating a function, which took in inputs of the updated pendulum position. A special effect was created to indicate when the impulse was applied to the system by highlighting the cart in red.
- The FuncAnimation library was used to animate the frames.

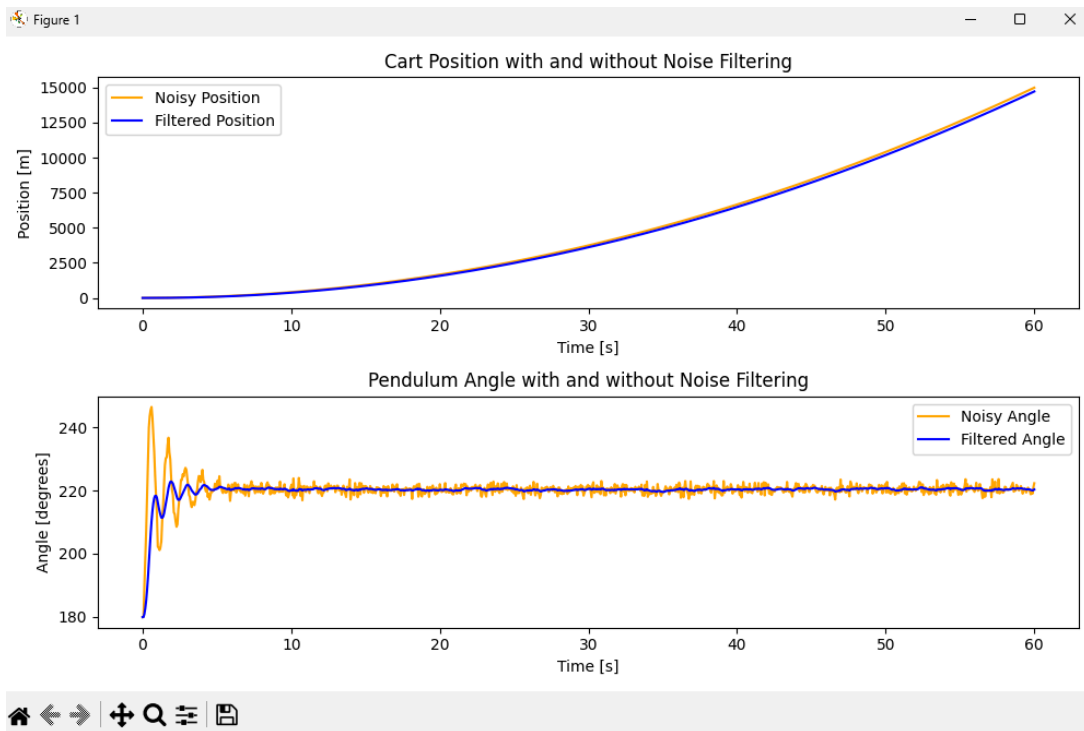


Figure 2.4: Graph of Inverted Pendulum representing Position vs. Time and Angle vs. Time

### 2.3.3 Simulated Modeling with Controller

#### PID

The code of the system using the PID controller is explained below:

- 1) Set up of PID Controller
  - The code started with importing the relevant libraries, numpy for computations, matplotlib for graphing visualizations, scipy to solve the nonlinear equations, place\_poles to compute the state space control design, and FuncAnimation to run the animation of the system.
  - Parameters were defined for the mass of cart and pendulum, length of pendulum, gravity and damping (which considered both friction and air resistance).

- The nonlinear equations of motion were defined as shown above and calculated within a function named equations
  - A class was created which defined the PID controller.  $K_p$ ,  $K_i$  and  $K_d$  were initialized. The method to compute the control output based on the setpoint and measured value of the system was done through a function within the class.
    - The function took in error between the setpoint of 180 degrees and the measured value. Then the proportional, integral, derivative values were outputted and returned.
- 2) System Behavior before and after impulse
- The simulation was set to run for 60 seconds with initial conditions of no displacement for cart position, no velocity for cart, 180 degrees for pendulum angle, no angular velocity for pendulum.
  - The PID gains for the pendulum angle control was defined.
  - An empty array was made to store the impulse force applied during the simulation.
  - The setpoint of the desired angle of the pendulum was set to 180 degrees- notating the upright position.
- 3) Simulation of the Dynamics of the System
- A for loop was set which ran through the 60 seconds as set for the simulation
  - Checked if the impulse time was within the current time window and applied the force at a magnitude of 5N. The force was updated to the force array.
  - At the start of the simulation the measurements were provided with noise
  - If no impulse was applied then the force array returned 0.
  - Over the time array chosen (0-60 seconds) the nonlinear equations were used to define the motion with the effect of the external force.
- 4) Animation and Graphing
- The simulation parameters were restricted within the displacement of -1.0 and 1.0 to prevent the cart from going off the screen
  - The graphing parameters were set by defining the axis labels, line colors, and legend.
  - A function was created to obtain the pendulum position in x,y coordinates
  - The cart and pendulum were created by creating objects which defined the characteristics of the system.
  - The animation updates were done through creating a function, which took in inputs of the updated pendulum position. A special effect was created to indicate when the impulse was applied to the system by highlighting the cart in red.

- The FuncAnimation library was used to animate the frames.

### Pole Placement

To derive the pole placement controller, the state space equations were used for the system. The state space equations reflect the system's natural tendencies without any control input using a matrix. Another matrix represents how inputs affect the system.

Using the linearized equations, the following matrix called the state vector was defined:

$$x_1 = x$$

$$x_2 = \dot{x}$$

$$x_3 = \theta$$

$$x_4 = \dot{\theta}$$

Where,

$$\dot{x} = Ax + Bu$$

The matrices are defined as follows:

Matrix A:

|          |               |                 |          |
|----------|---------------|-----------------|----------|
| $\theta$ | 1             | $\theta$        | $\theta$ |
| $\theta$ | $-b/(m1+m2)$  | $m2g/m1$        | $\theta$ |
| $\theta$ | $\theta$      | $\theta$        | 1        |
| $\theta$ | $-b/l(m1+m2)$ | $-(m1+m2)g/lm1$ | $\theta$ |

Matrix B:

|              |
|--------------|
| $\theta$     |
| $1/(m1+m2)$  |
| $\theta$     |
| $1/l(m1+m2)$ |

The feedback equation is presented as:

$$u = -Kx$$

Where,

$$K = \text{feedback gain response}$$

The code of the system using the pole placement controller is explained below:

### 1) Set up of Pole Placement Controller

- The code started with importing the relevant libraries, numpy for computations, matplotlib for graphing visualizations, scipy to solve the nonlinear equations, place\_poles to compute the state space control design, and FuncAnimation to run the animation of the system.
- Parameters were defined for the mass of cart and pendulum, length of pendulum, gravity and damping (which considered both friction and air resistance).
- The state space equations as shown above, were defined.
- The desired pole locations of the system were set- this refers to  $x$  in the feedback equation.
- The feedback gain response matrix was calculated and printed for the user.
- The state vector was then defined.

### 2) System Behavior before and after impulse

- Once the impulse was applied at  $t=2$  seconds the control input  $u$  was sent to the feedback loop
- The equations of motion for the inverted pendulum system were defined as shown above, these were calculated and returned to the state vector for integration
- The simulation was set to run for 60 seconds with initial conditions of no displacement for cart position, no velocity for cart, 180 degrees for pendulum angle, no angular velocity for pendulum
- An empty array was made to store the impulse force applied during the simulation
- The setpoint of the desired angle of the pendulum was set to 180 degrees- notating the upright position

### 3) Simulation of the Dynamics of the System

- A for loop was set which ran through the 60 seconds as set for the simulation
- Checked if the impulse time was within the current time window and applied the force at a magnitude of 5N. The force was updated to the force array.
- At the start of the simulation the measurements were provided with noise
- If no impulse if applied then force array was return 0
- Over the time array chosen (0-60 seconds) the nonlinear equations were used to define the motion with the effect of the external force

### 4) Animation and Graphing

- The simulation parameters were restricted within the displacement of -0.8 and 0.8 to prevent the cart from going off the screen

- The graphing parameters were set by defining the axis labels, line colors, and legend.
- A function was created to obtain the pendulum position in x,y coordinates
- The cart and pendulum were created by creating objects which defined the characteristics of the system.
- The animation updates were done through creating a function, which took in inputs of the updated pendulum position. A special effect was created to indicate when the impulse was applied to the system by highlighting the cart in red.
- The FuncAnimation library was used to animate the frames.

### 2.3.4 Benchmarking and Performance Metrics

The following graphs capture the angle vs. time of the pendulum using the two different controller systems:

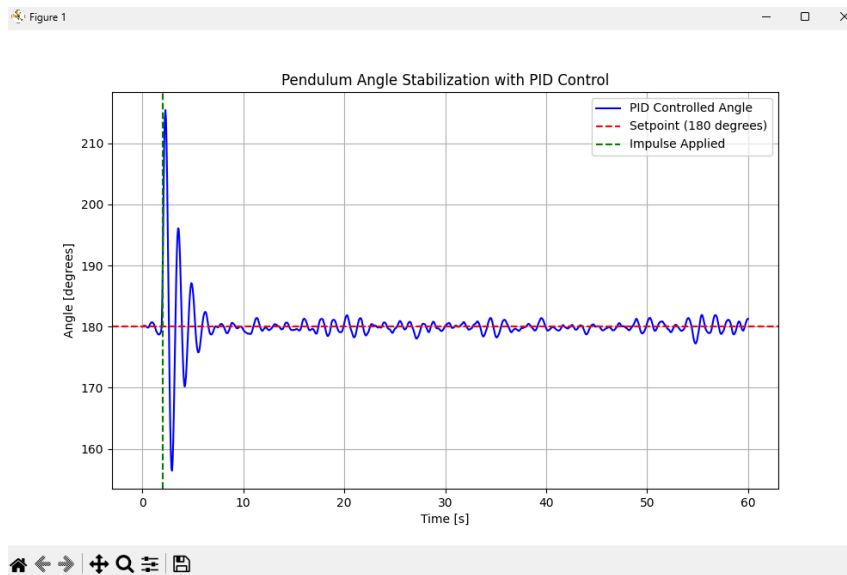


Figure 2.5: Graph of angle vs. time with PID Control

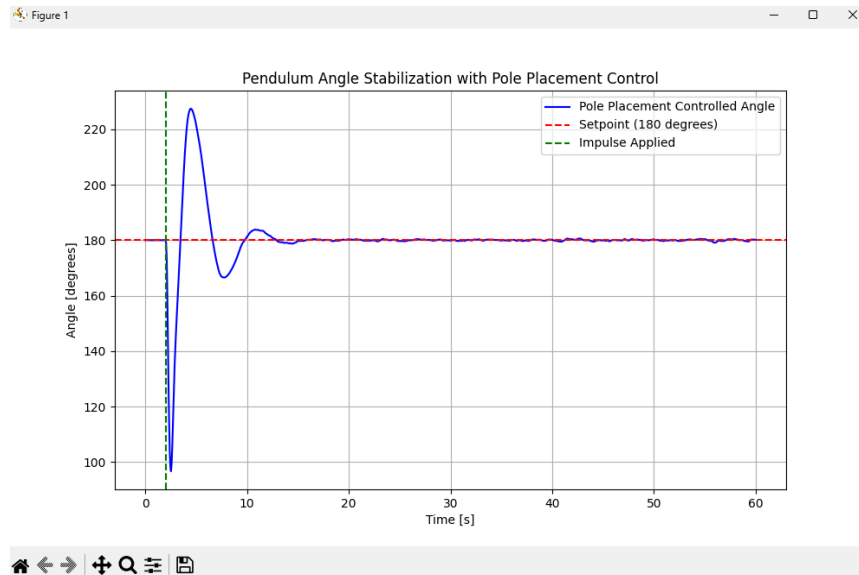


Figure 2.6 : Graph of angle vs. time with Pole Placement Control

The factors chosen to be analyzed for this system to judge the performance metrics were: settling time, overshoot, and robustness to noise.

**Settling Time:** The settling time for the PID controller was faster- seemingly settling within 5 seconds after the impulse was applied. Whereas, the pole placement controller settled around 15 seconds after the impulse was applied.

**Overshoot:** Similarly to the settling time, the overshoot was higher within the pole placement system than the PID system. The overshoot in the pole placement system was around 240 degrees, whereas the PID had an overshoot at around 215 degrees. However the PID controller oscillates at a much more frequent rate compared to the pole placement controller. Therefore, the PID controller also oscillates less but faster than the pole placement controller.

**Robustness to noise:** Comparing both the graphs, as both systems were exposed to noise, the PID controller was more sensitive to the noise than the pole placement controller. The movement due to noise in the pendulum caused more oscillations and stability issues when using the PID controller. This was seen once the system stabilized- as it had trouble maintaining the angle at the setpoint and kept oscillating a couple degrees. In comparison to the pole placement controller which also oscillated, however, the angle of oscillation was much less than the PID controller.



### 3.0 CONCLUSION & RECOMMENDATIONS

In this project, the inverted pendulum system was successfully modeled using SysML and simulated using Python. Three (3) different scenarios were simulated: the system under environmental factors, system controlled by a PID controller and pole placement controller. The highlights of the system were seen as faster stabilization times depending on overshoot and settling time was achieved by the PID controller, the pole placement controller provided a smoother response with better handling of noise

The following recommendations were made for the project:

Different filtration techniques: Implementing more advanced filtering techniques, such as a Kalman filter, to further reduce the impact of sensor noise.

Addition of other real life disturbances: Incorporating various real-life disturbances, such as uneven surfaces, to evaluate the robustness of the control strategies under more challenging conditions.

Sensitivity Analysis: Looking into variations in system parameters (e.g., mass, length of the pendulum, and damping coefficients) affect system performance. This analysis would help identify parameters for optimal control.

### 4.0 REFERENCES

- [1] "Inverted pendulum," Wikipedia, [https://en.wikipedia.org/wiki/Inverted\\_pendulum](https://en.wikipedia.org/wiki/Inverted_pendulum) (accessed Oct. 31, 2024).
- [2] J. Fišer and P. Zítek, "PID controller tuning via dominant pole placement in comparison with Ziegler-Nichols tuning," IFAC-PapersOnLine, <https://www.sciencedirect.com/science/article/pii/S240589631932083X> (accessed Oct. 31, 2024).
- [3] PESTLEWeb England, "SysML in Eclipse Papyrus," YouTube, [https://www.youtube.com/watch?v=W07ku7JIH5E&list=PLzwHRu\\_a7pYtDtw3Ws7t73P6TrcDrGnMA](https://www.youtube.com/watch?v=W07ku7JIH5E&list=PLzwHRu_a7pYtDtw3Ws7t73P6TrcDrGnMA) (accessed Oct. 31, 2024).
- [4] M. Jibril, M. Tadese, and F. Bogale, "Stability Analysis of an inverted pendulum on a cart with the presence of restoring and frictional forces disturbance using observer based and full state feedback H2 controllers," [v1] | Preprints.org, <https://www.preprints.org/manuscript/202010.0312/v1> (accessed Oct. 31, 2024).

[5] Admin, “Equations of motion of a pendulum on a cart by using Lagrange’s equations,” Fusion of Engineering Control Coding Machine Learning and Science, <https://aleksandarhaber.com/equations-of-motion-of-a-pendulum-on-a-cart-by-using-lagrange-equations/> (accessed Oct. 31, 2024).

## 5.0 APPENDIX

### 5.1 Task 2- System Exposure to Environmental Factors

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
from matplotlib.animation import FuncAnimation

# Parameters
M = 0.5      # mass of the cart in kg
m = 0.1      # mass of the pendulum in kg
L = 0.4      # length of the pendulum in meters
g = 9.81     # gravity in m/s^2
b = 0.5      # damping coefficient

# Nonlinear equations of motion
def equations(y, t, F):
    x, x_dot, theta, theta_dot = y
    sin_theta = np.sin(theta)
    cos_theta = np.cos(theta)

    # Nonlinear equations of motion with damping
    x_ddot = (F + m * L * theta_dot**2 * sin_theta) / (M + m)
    theta_ddot = (g * sin_theta - cos_theta * x_ddot - b * theta_dot) / L

    return [x_dot, x_ddot, theta_dot, theta_ddot]

# Initial conditions: cart position (x), cart velocity (x_dot), pendulum
# angle (theta), pendulum angular velocity (theta_dot)
y0 = [0.0, 0.0, np.pi, 0.0] # Start with 180 degrees angle (upright)

# Time array
t = np.linspace(0, 60, 1000)
```

```

# External Force
F = 5
solution = odeint(equations, y0, t, args=(F,)) #Solve nonlinear equations

# Extract results
x, x_dot, theta, theta_dot = solution.T

# Simulate sensor noise
def add_sensor_noise(data, noise_level=0.02):
    noise = np.random.normal(0, noise_level, data.shape)
    return data + noise

# Add noise to position and angle readings
x_noisy = add_sensor_noise(x)
theta_noisy = add_sensor_noise(theta)

# Low-pass filter
def low_pass_filter(data, alpha=0.1):
    filtered_data = np.zeros_like(data)
    filtered_data[0] = data[0] # Initialize with first value
    for i in range(1, len(data)):
        filtered_data[i] = alpha * data[i] + (1 - alpha) *
filtered_data[i-1]
    return filtered_data

# Filter the noisy measurements
x_filtered = low_pass_filter(x_noisy)
theta_filtered = low_pass_filter(theta_noisy)

# Visualization: compare the results with and without filtering
plt.figure(figsize=(10, 6))

plt.subplot(2, 1, 1)
plt.plot(t, x_noisy, label="Noisy Position", color='orange')
plt.plot(t, x_filtered, label="Filtered Position", color='blue')
plt.title("Cart Position with and without Noise Filtering")
plt.xlabel("Time [s]")
plt.ylabel("Position [m]")
plt.legend()

```

```

plt.subplot(2, 1, 2)
plt.plot(t, np.degrees(theta_noisy), label="Noisy Angle", color='orange')
plt.plot(t, np.degrees(theta_filtered), label="Filtered Angle",
color='blue')
plt.title("Pendulum Angle with and without Noise Filtering")
plt.xlabel("Time [s]")
plt.ylabel("Angle [degrees]")
plt.legend()

plt.tight_layout()
plt.show()

# Animation: simulate the cart and pendulum motion
def get_pendulum_coords(x, theta):
    pendulum_x = x + L * np.sin(theta)
    pendulum_y = -L * np.cos(theta)
    return pendulum_x, pendulum_y

# Create the figure and axis for animation
fig, ax = plt.subplots(figsize=(8, 6))
ax.set_xlim(-1, 1) # Set limits for x-axis
ax.set_ylim(-0.6, 0.6) # Set limits for y-axis

# Create objects representing the cart and pendulum
cart_width = 0.2
cart_height = 0.1
cart = plt.Rectangle((-cart_width / 2, -cart_height / 2), cart_width,
cart_height, fc='blue')
pendulum_line, = ax.plot([], [], 'o-', lw=2, color='red')

# Add cart to the plot
ax.add_patch(cart)

# Initialize the animation
def init():
    cart.set_xy((-cart_width / 2, -cart_height / 2)) # Set cart to the
initial position
    pendulum_line.set_data([], [])
    return cart, pendulum_line

```

```

# Update function for the animation
def update(frame):
    # Update cart position with filtered measurement
    cart_position = np.clip(x_filtered[frame] - cart_width / 2, -0.8, 0.8)
# Restrict cart position within x limits
    cart.set_x(cart_position)

    # Update pendulum position with filtered measurement
    pendulum_x, pendulum_y = get_pendulum_coords(cart_position +
cart_width / 2, theta_filtered[frame])
    pendulum_line.set_data([cart_position + cart_width / 2, pendulum_x],
[0, pendulum_y])

    return cart, pendulum_line

# Animate the motion
ani = FuncAnimation(fig, update, frames=len(t), init_func=init, blit=True,
interval=20)

# Show the animation
plt.title('Pendulum and Cart Simulation with Sensor Noise and Filtering')
plt.show()

```

## 5.2 Task 3- PID Controller

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
from matplotlib.animation import FuncAnimation

M = 0.5    # mass of the cart in kg
m = 0.1    # mass of the pendulum in kg
L = 0.4    # length of the pendulum in meters
g = 9.81   # gravity in m/s^2
b = 0.5    # damping coefficient

def equations(y, t, F_func, t_array):
    x, x_dot, theta, theta_dot = y
    sin_theta = np.sin(theta)

```

```

cos_theta = np.cos(theta)

# Get the control force for the current time step
F = np.interp(t, t_array, F_func)

x_ddot = (F + m * L * theta_dot**2 * sin_theta) / (M + m)
theta_ddot = (g * sin_theta - cos_theta * x_ddot - b * theta_dot) / L

return [x_dot, x_ddot, theta_dot, theta_ddot]

# PID Controller
class PIDController:
    def __init__(self, Kp, Ki, Kd):
        self.Kp = Kp
        self.Ki = Ki
        self.Kd = Kd
        self.prev_error = 0
        self.integral = 0

    def compute(self, setpoint, measured_value, dt):
        error = setpoint - measured_value
        self.integral += error * dt
        derivative = (error - self.prev_error) / dt
        output = self.Kp * error + self.Ki * self.integral + self.Kd *
derivative
        self.prev_error = error
        return output

t = np.linspace(0, 60, 1000)

y0 = [0.0, 0.0, np.pi, 0.0]

# Simulate PID control
Kp_theta, Ki_theta, Kd_theta = 5.0, 0.1, 1.0 # PID gains for pendulum
angle
pid_theta = PIDController(Kp_theta, Ki_theta, Kd_theta)

F_pid = []
setpoint_theta = np.pi # Upright

```

```

# Simulate dynamics
for i in range(len(t)):
    dt = t[i] - t[i-1] if i > 0 else 0.01

    # Apply impulse force at 2 seconds
    if 1.9 <= t[i] < 2.1:
        impulse_force = 5.0 # Magnitude of the impulse force
        F_pid.append(impulse_force)

        if i == 0:
            y0[2] = np.random.normal(0, 0.05) # Small noise added
        else:
            control_theta = pid_theta.compute(setpoint_theta, y0[2], dt)
            noise = np.random.normal(0, 0.1) # Add noise to the control force
            F_pid.append(control_theta + noise)

# Solve nonlinear equations
solution_pid = odeint(equations, y0, t, args=(F_pid, t))
x_pid, x_dot_pid, theta_pid, theta_dot_pid = solution_pid.T

# Keep the cart within the screen limits
x_pid = np.clip(x_pid, -0.8, 0.8)

# Visualization of the PID controlled angle
plt.figure(figsize=(10, 6))
plt.plot(t, np.degrees(theta_pid), label="PID Controlled Angle",
color='blue')
plt.title("Pendulum Angle Stabilization with PID Control")
plt.xlabel("Time [s]")
plt.ylabel("Angle [degrees]")
plt.axhline(180, color='red', linestyle='--', label="Setpoint (180
degrees)")
plt.axvline(2, color='green', linestyle='--', label="Impulse Applied")
plt.legend()
plt.grid(True)
plt.show()

# Animation: simulate the cart and pendulum motion
def get_pendulum_coords(x, theta):
    """Calculate the position of the pendulum end."""

```

```

pendulum_x = x + L * np.sin(theta)
pendulum_y = -L * np.cos(theta)
return pendulum_x, pendulum_y

# Create the figure and axis for animation
fig, ax = plt.subplots(figsize=(8, 6))
ax.set_xlim(-1, 1) # limits for x-axis
ax.set_ylim(-0.6, 0.6) # limits for y-axis

# Create objects representing the cart and pendulum
cart_width = 0.2
cart_height = 0.1
cart = plt.Rectangle((-cart_width / 2, -cart_height / 2), cart_width,
cart_height, fc='blue')
pendulum_line, = ax.plot([], [], 'o-', lw=2, color='red')

# Add cart to the plot
ax.add_patch(cart)

# Initialize the animation
def init():
    """Initialize the cart and pendulum."""
    cart.set_xy((-cart_width / 2, -cart_height / 2)) # Set cart to the
initial position
    pendulum_line.set_data([], [])
    return cart, pendulum_line

# Update function for the animation
def update(frame):
    """Update the cart and pendulum position for each frame."""
    # Update cart position
    cart.set_x(x_pid[frame] - cart_width / 2)

    # Update pendulum position
    pendulum_x, pendulum_y = get_pendulum_coords(x_pid[frame],
theta_pid[frame])
    pendulum_line.set_data([x_pid[frame], pendulum_x], [0, pendulum_y])

    # Highlight the impulse application
    if 1.9 <= t[frame] < 2.1:

```



```

        cart.set_edgecolor('red')
        cart.set_linewidth(3)
    else:
        cart.set_edgecolor('blue')
        cart.set_linewidth(1)

    return cart, pendulum_line

# Animate the motion
ani = FuncAnimation(fig, update, frames=len(t), init_func=init, blit=True,
interval=20)

# Show the animation
plt.title('Pendulum and Cart Simulation with PID Control')
plt.show()

```

### 5.3 Task 3- Pole Placement Controller

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
from scipy.signal import place_poles
from matplotlib.animation import FuncAnimation

M = 0.5    # mass of the cart in kg
m = 0.1    # mass of the pendulum in kg
L = 0.4    # length of the pendulum in meters
g = 9.81   # gravity in m/s^2
b = 0.5    # damping coefficient

# State-space matrices A and B
A = np.array([[0, 1, 0, 0],
              [0, -b/(M + m), m*g/M, 0],
              [0, 0, 0, 1],
              [0, -b/(L * (M + m)), -(M + m) * g/(L * M), 0]])

B = np.array([[0],
              [1/(M + m)],
              [0],

```

```

        [1/(L * (M + m))]])

# Desired pole locations
desired_poles = np.array([-2.0, -2.5, -3.0, -3.5])

# Calculate the gain matrix K using pole placement
place_solution = place_poles(A, B, desired_poles)
K = place_solution.gain_matrix
print("Pole Placement Gain Matrix K:", K)#print K

def equations_pole_placement(y, t, F_func, t_array):
    x, x_dot, theta, theta_dot = y
    state = np.array([[x], [x_dot], [theta], [theta_dot]])

    # Get the control input with noise for the current time step
    control_input = -K @ state # Control input based on the state

    if t < 2.0:
        u = 0
    else:
        u = control_input.item() + np.interp(t, t_array, F_func)

    # Nonlinear equations of motion
    sin_theta = np.sin(theta)
    cos_theta = np.cos(theta)
    x_ddot = (u + m * L * theta_dot**2 * sin_theta) / (M + m)
    theta_ddot = (g * sin_theta - cos_theta * x_ddot - b * theta_dot) / L
    return [x_dot, x_ddot, theta_dot, theta_ddot]

t = np.linspace(0, 60, 1000)

y0 = [0.0, 0.0, np.pi, 0.0]

F_pp = []
setpoint_theta = np.pi

# Simulate
for i in range(len(t)):
    dt = t[i] - t[i-1] if i > 0 else 0.01

```

```

    if 1.9 <= t[i] < 2.1:
        impulse_force = 5.0
        F_pp.append(impulse_force)

    if i == 0:
        y0[2] = np.random.normal(0, 0.05)
    else:
        # Calculate control input using pole placement controller
        control_input = -K @ np.array([[y0[0]], [y0[1]], [y0[2]],
[y0[3]]]) # Control input based on state

        # Add noise to the control force
        noise = np.random.normal(0, 0.1) # Simulate noise
        F_pp.append(control_input.item()+noise)

# Simulate nonlinear dynamics with Pole Placement control
solution_pp = odeint(equations_pole_placement, y0, t, args=(F_pp, t))
x_pp, x_dot_pp, theta_pp, theta_dot_pp = solution_pp.T

x_pp = np.clip(x_pp, -0.8, 0.8)

plt.figure(figsize=(10, 6))
plt.plot(t, np.degrees(theta_pp), label="Pole Placement Controlled Angle",
color='blue')
plt.title("Pendulum Angle Stabilization with Pole Placement Control")
plt.xlabel("Time [s]")
plt.ylabel("Angle [degrees]")
plt.axhline(180, color='red', linestyle='--', label="Setpoint (180
degrees)")
plt.axvline(2, color='green', linestyle='--', label="Impulse Applied")
plt.legend()
plt.grid(True)
plt.show()

def get_pendulum_coords(x, theta):
    pendulum_x = x + L * np.sin(theta)
    pendulum_y = -L * np.cos(theta)
    return pendulum_x, pendulum_y

fig, ax = plt.subplots(figsize=(8, 6))

```

```

ax.set_xlim(-1, 1)
ax.set_ylim(-0.6, 0.6)

cart_width = 0.2
cart_height = 0.1
cart = plt.Rectangle((-cart_width / 2, -cart_height / 2), cart_width,
cart_height, fc='blue')
pendulum_line, = ax.plot([], [], 'o-', lw=2, color='red')

ax.add_patch(cart)

def init():
    cart.set_xy((-cart_width / 2, -cart_height / 2))
    pendulum_line.set_data([], [])
    return cart, pendulum_line

def update(frame):
    cart.set_x(x_pp[frame] - cart_width / 2)

    pendulum_x, pendulum_y = get_pendulum_coords(x_pp[frame],
theta_pp[frame])
    pendulum_line.set_data([x_pp[frame], pendulum_x], [0, pendulum_y])

    if 1.9 <= t[frame] < 2.1:
        cart.set_edgecolor('red')
        cart.set_linewidth(3)
    else:
        cart.set_edgecolor('blue')
        cart.set_linewidth(1)

    return cart, pendulum_line

ani = FuncAnimation(fig, update, frames=len(t), init_func=init, blit=True,
interval=20)

plt.title('Pendulum and Cart Simulation with Pole Placement Control')
plt.show()

```